Lab 3

Dockerfiles and Application Files

Show app.py content:

app.py E:\lab3\app.py

```python
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Lab 3"

@app.route("/health")
def health():
    return "ok"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Dockerfile.basic:

Dockerfile.basic E:\lab3\Dockerfile.basic

```dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Dockerfile.multistage:

Dockerfile.multistage E:\lab3\Dockerfile.multistage

```
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --prefix=/install -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

requirements.txt:

```
PS E:\lab3> Get-Content E:\lab3\requirements.txt
flask==3.0.0
```

Kubernetes YAML Files

deployment.yaml:

deployment.yaml E:\lab3\deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab3-web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: lab3-web
  template:
    metadata:
      labels:
        app: lab3-web
    spec:
      nodeSelector:
        node-role: general
      containers:
      - name: web
        image: lab3-multi
        imagePullPolicy: Never
        ports:
        - containerPort: 5000
```

service.yaml:

service.yaml E:\lab3\service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: lab3-web-svc
spec:
  selector:
    app: lab3-web
  ports:
  - port: 80
    targetPort: 5000
  type: ClusterIP
```

Screenshots of Pods, Services, Deployment

Cluster running:

```
PS E:\lab3> kubectl get nodes
NAME                        STATUS   ROLES           AGE   VERSION
lab3-cluster-control-plane  Ready    control-plane   11h   v1.29.2
```

Show both images built:

```
PS E:\lab3> docker image ls
                                                  i Info →   U  In Use
IMAGE                        ID             DISK USAGE   CONTENT SIZE   EXTRA
gcr.io/etcd-development/etcd:v3.5.0
                             28759af54acd        156MB         40.6MB
kindest/node@sha256:51a1434a5397193442f0be2a297b488b6c919ce8a3931be0ce822606
ea5ca245
                             51a1434a5397       1.39GB          409MB    U
lab3-basic:latest            ddf498f0aa38        199MB           49MB
lab3-multi:latest            66c872258e96        184MB         44.7MB
lab4-order-service:latest    073ed6a5fcbd        202MB         49.2MB    U
lab4-product-service:latest
                             d2980029f90e        197MB         48.1MB    U
redis:latest                 832d7785830f        204MB         55.3MB
ubuntu:22.04                 962f6cadeae0        119MB         31.7MB
PS E:\lab3>
```

Activate Windows
Go to Settings to activate Windows.

Show docker history (layer caching):

```
PS E:\lab3> docker history lab3-basic
IMAGE          CREATED        CREATED BY
  SIZE      COMMENT
ddf498f0aa38   12 hours ago   CMD ["python" "app.py"]
  0B        buildkit.dockerfile.v0
<missing>      12 hours ago   EXPOSE [5000/tcp]
  0B        buildkit.dockerfile.v0
<missing>      12 hours ago   COPY app.py . # buildkit
  12.3kB    buildkit.dockerfile.v0
<missing>      12 hours ago   RUN /bin/sh -c pip install -r requirements.t…
  16.5MB    buildkit.dockerfile.v0
<missing>      12 hours ago   COPY requirements.txt . # buildkit
  12.3kB    buildkit.dockerfile.v0
<missing>      12 hours ago   WORKDIR /app
  8.19kB    buildkit.dockerfile.v0
<missing>      4 days ago     CMD ["python3"]
  0B        buildkit.dockerfile.v0
<missing>      4 days ago     RUN /bin/sh -c set -eux;  for src in idle3 p…
  16.4kB    buildkit.dockerfile.v0
<missing>      4 days ago     RUN /bin/sh -c set -eux;   savedAptMark="$(a…
  41.3MB    buildkit.dockerfile.v0
<missing>      4 days ago     ENV PYTHON_SHA256=c08bc65a81971c1dd578318282…
  0B        buildkit.dockerfile.v0
<missing>      4 days ago     ENV PYTHON_VERSION=3.12.13
  0B        buildkit.dockerfile.v0
<missing>      4 days ago     ENV GPG_KEY=7169605F62C751356D054A26A821E680…
  0B        buildkit.dockerfile.v0
<missing>      4 days ago     RUN /bin/sh -c set -eux;  apt-get update;  a…
  4.94MB    buildkit.dockerfile.v0
<missing>      4 days ago     ENV LANG=C.UTF-8
  0B        buildkit.dockerfile.v0
<missing>      4 days ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…
  0B        buildkit.dockerfile.v0
<missing>      5 days ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…
  87.4MB    debuerreotype 0.17
PS E:\lab3> docker history lab3-multi
IMAGE          CREATED        CREATED BY
  SIZE      COMMENT
66c872258e96   12 hours ago   CMD ["python" "app.py"]
  0B        buildkit.dockerfile.v0
```

Activate Windows
Go to Settings to activate Windows.

```
PS E:\lab3> docker history lab3-multi
IMAGE          CREATED        CREATED BY
  SIZE      COMMENT
66c872258e96   12 hours ago   CMD ["python" "app.py"]
  0B        buildkit.dockerfile.v0
<missing>      12 hours ago   EXPOSE [5000/tcp]
  0B        buildkit.dockerfile.v0
<missing>      12 hours ago   COPY app.py . # buildkit
  12.3kB    buildkit.dockerfile.v0
<missing>      12 hours ago   COPY /install /usr/local # buildkit
  5.51MB    buildkit.dockerfile.v0
<missing>      12 hours ago   WORKDIR /app
  8.19kB    buildkit.dockerfile.v0
<missing>      4 days ago     CMD ["python3"]
  0B        buildkit.dockerfile.v0
<missing>      4 days ago     RUN /bin/sh -c set -eux;  for src in idle3 p…
  16.4kB    buildkit.dockerfile.v0
<missing>      4 days ago     RUN /bin/sh -c set -eux;   savedAptMark="$(a…
  41.3MB    buildkit.dockerfile.v0
<missing>      4 days ago     ENV PYTHON_SHA256=c08bc65a81971c1dd578318282…
  0B        buildkit.dockerfile.v0
<missing>      4 days ago     ENV PYTHON_VERSION=3.12.13
  0B        buildkit.dockerfile.v0
<missing>      4 days ago     ENV GPG_KEY=7169605F62C751356D054A26A821E680…
  0B        buildkit.dockerfile.v0
<missing>      4 days ago     RUN /bin/sh -c set -eux;  apt-get update;  a…
  4.94MB    buildkit.dockerfile.v0
<missing>      4 days ago     ENV LANG=C.UTF-8
  0B        buildkit.dockerfile.v0
<missing>      4 days ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…
  0B        buildkit.dockerfile.v0
<missing>      5 days ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…
  87.4MB    debuerreotype 0.17
```

Activate Windows
Go to Settings to activate Windows.

Show pods running:

```
PS E:\lab3> kubectl get pods -o wide
NAME                        READY   STATUS             RESTARTS         AGE
  IP           NODE                          NOMINATED NODE   READINESS GATES
lab3-web-585d88dd9c-n9snl   0/1     CrashLoopBackOff   8 (4m49s ago)    20m
  10.244.0.9   lab3-cluster-control-plane   <none>           <none>
lab3-web-585d88dd9c-sfzfs   0/1     CrashLoopBackOff   9 (2m43s ago)    23m
  10.244.0.8   lab3-cluster-control-plane   <none>           <none>
lab3-web-5c5f649ddc-8lhdr   0/1     CrashLoopBackOff   11 (2m31s ago)   34m
  10.244.0.7   lab3-cluster-control-plane   <none>           <none>
lab3-web-5c5f649ddc-tpkcg   0/1     CrashLoopBackOff   11 (2m53s ago)   34m
  10.244.0.6   lab3-cluster-control-plane   <none>           <none>
```

Show services:

```
PS E:\lab3> kubectl get services
NAME           TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)   AGE
kubernetes     ClusterIP   10.96.0.1     <none>        443/TCP   11h
lab3-web-svc   ClusterIP   10.96.189.5   <none>        80/TCP    34m
```

Activate Windows
Go to Settings to activate Windows.

Show deployment details:

```
PS E:\lab3> kubectl get deployments
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
lab3-web    0/3     2            0           34m
```

```
PS E:\lab3> kubectl describe deployment lab3-web
Name:                   lab3-web
Namespace:              default
CreationTimestamp:      Sun, 26 Apr 2026 14:20:59 +0300
Labels:                 <none>
Annotations:            deployment.kubernetes.io/revision: 2
Selector:               app=lab3-web
Replicas:               3 desired | 2 updated | 4 total | 0 available | 4 un
available
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=lab3-web
  Containers:
   web:
    Image:         lab3-multi
    Port:          5000/TCP
    Host Port:     0/TCP
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  node-role=general
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      False   MinimumReplicasUnavailable
  Progressing    False   ProgressDeadlineExceeded
OldReplicaSets:  lab3-web-5c5f649ddc (2/2 replicas created)
NewReplicaSet:   lab3-web-585d88dd9c (2/2 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  35m   deployment-controller  Scaled up replica
set lab3-web-5c5f649ddc to 3
  Normal  ScalingReplicaSet  24m   deployment-controller  Scaled up replica
set lab3-web-585d88dd9c to 1
  Normal  ScalingReplicaSet  21m   deployment-controller  Scaled down replic
a set lab3-web-5c5f649ddc to 2 from 3
  Normal  ScalingReplicaSet  21m   deployment-controller  Scaled up replica
```

Activate Windows
Go to Settings to activate Windows.

Show node labels

```
PS E:\lab3> kubectl get nodes --show-labels
NAME                        STATUS   ROLES           AGE   VERSION   LABELS
lab3-cluster-control-plane   Ready    control-plane   11h   v1.29.2   beta.k
ubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64
,kubernetes.io/hostname=lab3-cluster-control-plane,kubernetes.io/os=linux,no
de-role.kubernetes.io/control-plane=,node-role=general
```

Before deleting pod:

```
PS E:\lab3> kubectl get pods
NAME                          READY   STATUS             RESTARTS        AGE
lab3-web-585d88dd9c-n9snl     0/1     CrashLoopBackOff   10 (3m6s ago)   29m
lab3-web-585d88dd9c-sfzfs     0/1     CrashLoopBackOff   11 (62s ago)    32m
lab3-web-5c5f649ddc-8lhdr     0/1     CrashLoopBackOff   13 (47s ago)    42m
lab3-web-5c5f649ddc-tpkcg     0/1     CrashLoopBackOff   13 (81s ago)    42m
PS E:\lab3>
```

Activate Windows
Go to Settings to activate Windows.

Question: Why does the container think its main process is PID 1 while the host sees a different PID? Explain the role of the PID namespace.?
Inside the container the main process appears as PID 1. On the host, Docker shows it as a much larger number (e.g. 1850). This is because of the PID namespace — each container gets its own isolated process ID space starting from 1. The host sees the real system PID, but inside the container the process believes it is the first and only process.
Which image is smaller? "lab3-multi (multi-stage) is smaller because the builder stage is discarded after copying only the installed packages — no build tools remain in the final image."
Which steps are cached if only app.py changes? "The COPY requirements.txt and RUN pip install steps are cached because requirements.txt didn't change. Only the COPY app.py step and everything after it re-runs."
Why does layer order matter? "Layers that change frequently (like app.py) should be placed last. Layers that rarely change (like installing dependencies) should be placed first so Docker can reuse them from cache and speed up builds."
Reflection question: Why is this different from hard-coding a machine name in traditional deployment models? How does declarative scheduling improve resilience and maintainability?
Hard-coding a machine name ties the deployment to one specific server — if that server fails, the application goes down. Declarative scheduling with node selectors tells Kubernetes what KIND of node to use, not which specific one. This improves resilience because Kubernetes can place pods on any matching node automatically. It also improves maintainability because you describe the desired state and let the scheduler handle placement decisions.
Watch the Pod state from another terminal using kubectl get pods -w. Question: Did Kubernetes restart the container in place or create a brand-new Pod? Explain the difference based on what you observe.
Hard-coding a machine name ties the deployment to one specific server — if that server fails, the application goes down. Declarative scheduling with node selectors tells Kubernetes what KIND of node to use, not which specific one. This improves resilience because Kubernetes can place pods on any matching node automatically. It also improves maintainability because you describe the desired state and let the scheduler handle placement decisions.

Reflection Questions

1. Why do namespaces alone not guarantee fair resource use?

Namespaces only isolate visibility a process in its own namespace can still consume 100% CPU or all available RAM. You need cgroups to actually enforce limits.

2. How do cgroups improve cluster stability?

They prevent one misbehaving container from starving others. Each container gets a guaranteed ceiling on CPU and memory, so the whole node stays stable.

3. Why is Docker image layering important for large-scale orchestration?

Shared layers are downloaded once and reused across many containers. In a cluster of 100 nodes pulling the same image, only the changed layers transfer — saving time and bandwidth dramatically.

4. What does Kubernetes mean by desired state?

A declaration of what you want "3 replicas of this container running." Kubernetes continuously watches actual state and takes actions to match it, whether that means creating, restarting, or rescheduling pods.

5. How is self-healing different from traditional manual operations?

Traditionally, an operator gets paged, logs in, and manually restarts a failed service. Kubernetes detects failure automatically and recovers in seconds with no human involved the system heals itself 24/7.

6. Why are readiness and liveness probes not interchangeable?

Liveness asks "is the container alive?" failure triggers a restart. Readiness asks "is the container ready to serve traffic?" — failure just removes it from the load balancer without restarting. A container can be alive but not yet ready (still loading data), so treating them the same would cause unnecessary restarts.

7. One limitation of a single-node kind cluster vs real multi-node cluster?

In kind, all pods land on one node so you cannot observe real scheduling decisions spreading across nodes, affinity rules pulling pods apart, or a node failure causing pods to reschedule elsewhere. It shows the API but not the real distributed behavior.